

Received XX Month, XXXX; revised XX Month, XXXX; accepted XX Month, XXXX; Date of publication XX Month, XXXX; date of current version 12 April, 2025.

Digital Object Identifier 10.1109/OJCOMS.2025.XXX

ML-Aided Dynamic BSR Periodicity Adjustment for Enhanced UL Scheduling in Cellular Systems

Nadezhda Chukhno¹, Salwa Saafi¹, and Sergey Andreev^{1,2}

¹Tampere Wireless Research Center, Tampere University, 33720 Tampere, Finland

²Department of Telecommunications, Brno University of Technology, 60190 Brno, Czech Republic

CORRESPONDING AUTHOR: N. Chukhno (e-mail: nadezda.chukhno@tuni.fi).

ABSTRACT Contemporary research has revealed a limitation in the Uplink (UL) Buffer Status Report (BSR) scheduling procedure – its reliance on outdated information. In addition, a significant limitation in current BSR implementations lies in their inflexibility. The 3rd Generation Partnership Project (3GPP) specifications constrain BSR periodicities to certain quantized values based on Quality of Service (QoS) requirements for various applications. For instance, applications demanding low latency may require very small BSR periodicities, resulting in substantial overhead due to frequent BSR reports. This may result in the wastage of network resources in case of a low BSR periodicity setting. Alternatively, a high BSR periodicity setting may lead packets to wait more at the user buffer and thus result in higher packet latencies. To address these limitations, we propose a framework that predicts time intervals between packet arrivals and subsequently adjusts the BSR periodicity according to the predicted traffic arrivals. The simulation results demonstrate that the proposed Machine Learning (ML)-aided BSR reporting provides flexibility in BSR periodicity adapted to the intensity of traffic arrival and converges to optimal periodicity depending on the mean traffic arrival rate.

INDEX TERMS Buffer status report, uplink scheduling, cellular networks, machine learning, uplink traffic prediction, wireless communications, machine learning for communications

I. INTRODUCTION AND BACKGROUND

A. Research Motivation

HUGE amounts of traffic data are generated owing to the ongoing advancements in cellular communications, which allow an increasing number of mobile devices to access the Internet [1], [2]. Traffic scenarios, including webpages, Internet-based applications, Uplink (UL) videos, and file transfers, account for a large share of traffic data. Moreover, while each cellular technology delivers notably higher throughput than the previous one, 5G promises a significant reduction in latency and support for Ultra Reliable Low Latency Communications (URLLC), ensuring more efficient data transfer scheduling. Thus, it is essential to study and improve the UL scheduling procedure and its main component – Buffer Status Report (BSR) [3].

BSR is a Medium Access Control (MAC)-layer feedback that allows the User Equipment (UE) to inform the Next Generation NodeB (gNB) that it has some data to transmit with the goal of obtaining a grant and sending the data. The 3rd Generation Partnership Project (3GPP) defines three

types of BSRs: regular BSR, periodic BSR, and padding BSR [3]. Regular BSR is sent when new data arrives in the UL buffer, and the new data has a higher priority than the one already waiting in the buffer; or when retransmission timer for BSR is expired and UE has some data to transmit. Periodic BSR is sent with a predefined periodicity (defined by the network), which controls how frequently the UE sends the BSR report (e.g., 'sf1', 'sf5', 'sf10', etc.). The 'sf' term in the BSR periodicity values, defined in the 3GPP specifications, denotes the subframe. For instance, 'sf1' means a BSR periodicity of 1 ms. Padding BSR is sent when the number of padding bits in a data message is larger than the size of the BSR, so that the padding bit space can be used to send the BSR.

While each BSR type has its own transmission mechanism, the three types of BSRs can coexist within the cellular systems [4]. Recently, several research works have pointed out a notable limitation in the periodic BSR mechanism – its inability to account for data that arrives between the transmission of the BSR and the delivery of UL data. As a

result, the newly arrived data must wait for the next UL transmission cycle instead of being transmitted immediately. This leads to an increased latency. To address this issue, several authors from both the academia and industry communities have explored statistical and Machine Learning (ML)-based approaches to predict future traffic arrivals, i.e., the number of packets expected to arrive at the UE during some time interval, to adjust BSR value accordingly.

For example, in [5], authors predict the data arrival rate determined as data volume indicated in current BSR divided by the difference between the serial number of the subframe of current and previous BSRs. While the authors provide the set of results regarding ML algorithms' accuracy, network-side metrics, such as latency or resource usage, are not investigated. Differently, in [6], a linear predictive model estimates the buffer occupancy at UE, which happens more frequently than BSR. This approach demonstrates enhancements in terms of latency and radio capacity utilization across a range of traffic patterns. However, a more general traffic source model rather than a constant bitrate over time should be incorporated into the UL schedulers for practical deployments.

Moreover, while there is a well-established field for analyzing aggregated network traffic, there is a notable research gap in understanding and assessing traffic at the individual users. To this end, in [7], [8], authors utilize ML algorithms to predict future packet arrivals and burst occurrences. Their works demonstrate that incorporating burst traffic arrival estimation into the decision-making module leads to a notable reduction in experienced latency, preventing delays for sensitive traffic. Similarly, in [10], the ML models are leveraged for dynamic Discontinuous Reception (DRX) parameter adaptation to user traffic. Namely, authors predict upcoming traffic (in terms of packet number), which guides the dynamic adjustment of DRX settings for energy- and cost-efficient performance. Although it effectively addresses a gap in cellular traffic prediction analysis, the study exhibits two limitations. Firstly, authors primarily emphasize traffic load prediction, a common focus in existing literature, neglecting the crucial aspect of predicting packet arrival times for informed DRX decisions. Secondly, the implementation design lacks depth, leaving uncertainties regarding how the network should handle UE-side data and what entity is responsible for predictions.

Another limitation in current periodic BSR implementations [3] neglected by the research community lies in their inflexibility. 3GPP specifications constrain BSR periodicities to certain quantized values based on Quality of Service (QoS) requirements for various applications with different QoS needs. The problem is that the packet interarrival period varies with time for certain applications (e.g., cloud gaming service). To ensure adequate quality of experience to the users, the QoS parameters (including BSR periodicity) during session initialization may be set ineffectively. For example, the BSR periodicity may be set to a very low

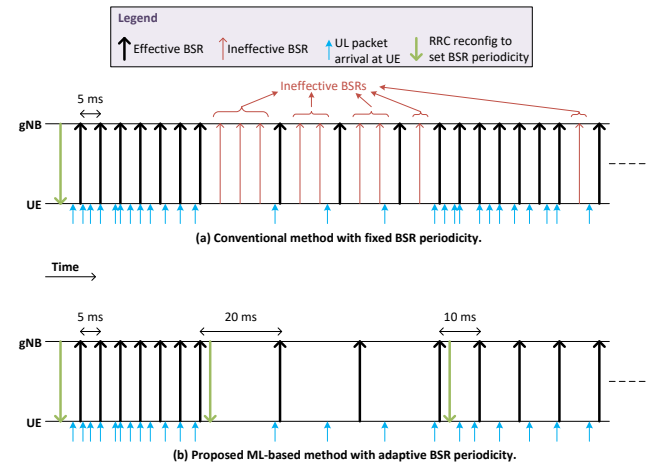


FIGURE 1. Illustration of (a) conventional 3GPP solution in comparison with (b) proposed solution.

value to limit the maximum tolerable latency of packets. This may result in wastage of network resources (in case of low BSR periodicity setting, i.e., more frequent BSR reports) and/or higher packet latencies (in case of high BSR periodicity setting, i.e., less frequent BSR reports). Moreover, such fixed BSR periodicity is not advantageous with the traffic that often requires dynamic and immediate resource allocation to meet its stringent reliability and latency demands (i.e., URLLC). The quantized values might not be sufficient to satisfy the application's requirements, especially when one periodicity is selected. In this work, we strive to fill this gap by proposing an adaptive BSR scheduling framework that predicts time intervals between packet arrivals and subsequently adjusts BSR periodicities defined in specifications based on the predicted traffic arrival patterns, thereby effectively managing network dynamics.

B. Contributions and Outline

As outlined earlier and summarized in Table 1, existing literature predominantly focuses on forecasting traffic load, yet it is equally crucial to anticipate the arrival time of the traffic. To the best of our knowledge, we are the first to predict the interval (in slots or milliseconds) between consecutive packet arrivals, leveraged for adaptive BSR scheduling. The ultimate goal is to provide a flexible scheduling framework and achieve satisfactory latency performance while reducing resource usage.

The novelty of the proposed solution is illustrated in Fig. 1. In the traditional method defined by 3GPP [3] (Fig. 1(a)), the BSR periodicity is fixed based on QoS requirements of the application, typically during the Packet Data Unit (PDU) session establishment. However, due to the variations in packet arrivals over time, several BSR transmissions may be ineffective. This problem can be resolved by adapting the BSR periodicity as illustrated in Fig. 1(b). Here, the ML-based prediction algorithm at the gNB automatically

TABLE 1. State-of-the-art traffic prediction.

Work	Problem	Prediction	UE/gNB side learning	Models	Dataset	Limitations
Literature						
[5]	BSR does not include newly arrived data during the interval between its sending and the UL data transmission, causing a delay for this data until the next round of UL transmission	Data arrival rate prediction calculated as data volume indicated in t -th BSR divided by the difference between the serial number of the subframe of t -th and $t-1$ -th BSRs	gNB side	LSTM, Random Forest, XGBoost	✗	Network-side performance, such as latency or resource usage, was not investigated (only ML accuracy)
[6]	gNB works on systematically outdated information about the buffer size, which generates delay spikes	Predictive estimation of the buffer size that is more frequently updated than the BSR	gNB side	Linear model (no ML)	✗	Proposed buffer size estimation algorithm models traffic source as being of constant bitrate over a time period (should be improved with a more general traffic model)
[7], [8]	Lack of research on the analysis of cellular traffic, especially user-level traffic. Aim is to control the level of available network resources	Traffic load prediction and classification	UE side data/ not specified where ML is implemented	LSTM, ARIMA, Random Forest	[9]	Implementation design is not specified (how the network should manage UE side data prediction)
[10]	Energy- and cost-efficient resource provisioning	Prediction of future traffic (number of packets), and based on this prediction, the best DRX is set dynamically	UE side data/ not specified where ML is implemented	LSTM, ARIMA, Random Forest	[9]	Implementation design is not specified. How can traffic prediction assist DRX? It is crucial to anticipate when packets will arrive rather than focusing solely on their size
Our Proposal						
Our work	(i) Conventional BSR mechanism works on outdated information. (ii) BSR periodicity is not flexible and cannot be adjusted according to the traffic, leading either to a waste of resources or increased latency	Predicting the time when the next packet arrives to the UE buffer	Two options for gNB side (algorithms 1, 2) and one for UE side prediction (algorithm 3)	LSTM, SVR, XGBoost, Random Forest	[9] and synthetic dataset	

triggers changes in BSR periodicity depending on the pattern of UL packet arrivals.

The contributions of this paper are as follows:

- We show that the packet interarrival times, representing the traffic arrival rate, can be accurately predicted and used for adaptive (yet periodic) BSR scheduling.
- We use ML-based method to dynamically adjust the BSR periodicity appropriately depending on the current variations in traffic interarrival times.
- We detail the implementation design of the proposal. Specifically, we provide three ways of handling the UE data for BSR periodicity prediction and, thereby, its adjustment to the UE traffic pattern by the gNB.

It is important to emphasize that the objective is not to design the optimal ML model for predicting traffic arrival

time, but rather to explore how ML can be utilized to enhance the BSR mechanism in general. We have selected conventional ML models as an initial approach to demonstrate the benefits of incorporating prediction into the conventional BSR mechanism. Naturally, more robust and advanced ML models can be considered as extensions of this work. For example, in [11], the discovery of sparsity and self-similarity in network traffic is leveraged to develop compact algorithms with high accuracy and low computational complexity and address the problem of frequent model re-training, while in [12], online-learning-based optimal predictive scheduling is proposed to deal with the unpredictability of data arrivals. Moreover, a universal zero-shot forecaster (Moirai) model from [13] eliminates the necessity for training, as it has already been pre-trained on numerous time-series datasets.

The remainder of this paper is organized as follows. In Section II, we outline the problem of BSR scheduling we attempt to resolve. Section III presents the proposed adaptive ML-aided BSR framework with three options for handling UE-sided data. In Section IV, we discuss the most common ML models used for traffic and packet arrival prediction, followed by the numerical results and outcomes derived from the framework's implementation provided in Section V. Conclusions and future work are discussed in the last Section VI.

II. PROBLEM FORMULATION

The current BSR implementation lacks flexibility, particularly concerning periodic reporting, and heavily depends on outdated information. In this section, we describe in detail the limitations we attempt to address in the BSR mechanism.

BSR procedure serves to inform gNB about data volume present at the UE's MAC entity and ready to be sent in UL. To initiate BSR transmission, the UE must estimate the amount of data it intends to transmit. This estimation relies on the data present in Radio Link Control (RLC) and Packet Data Convergence Protocol (PDCP) layers. Then, BSR is triggered by a periodic BSR timer (see Fig. 2), which controls how frequently the UE sends a periodic BSR [14]. The valid BSR periodicities the network can operate with according to 3GPP specifications are sf1, sf5, sf10, sf16, sf20, sf32, sf40, sf64, sf80, sf128, sf160, sf320, sf640, sf1280, sf2560, infinity.

In Fig. 2, we selected the periodicity of 5 ms. The periodic timer is a configurable parameter set by the network (gNB). It defines the interval at which the UEs check their buffer status and trigger a BSR report. Upon receiving the BSR, the gNB uses this information to efficiently allocate UL resources. It considers the buffer status reported by multiple UEs to dynamically schedule transmissions, ensuring that resources are allocated to UEs with pending data to be sent. The problem appears when the new packet arrives right after BSR report was sent. In this case, the newly arrived packet has to wait for the next periodic BSR timer to start the scheduling procedure, which introduces the delay. Furthermore, the periodic BSR report might be used when there are no packets available at the UE (see $t = 10$ ms in Fig. 2). This leads to resource wastage in UL.

Therefore, as can be seen in Fig. 2, the same BSR periodicity might not be suitable for all packets (and UEs), causing delays or waste of resources. For example, for applications or data streams with varying bit-rate characteristics that exhibit varying packet arrival rates, setting the BSR periodicity to a fixed value could be sub-optimal. On the one hand, small periodicity settings (such as 1, 5, and 10 ms) may cause unnecessarily large overheads during times of low packet arrival rates. On the other hand, large BSR periodicity settings may lead to high latencies.

In brief, the current BSR implementation exhibits two notable limitations: (i) its reliance on outdated information and

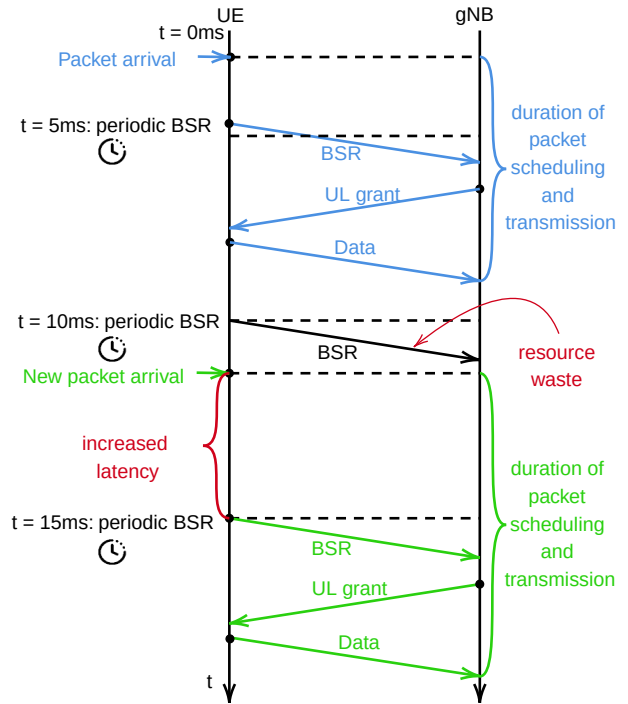


FIGURE 2. Visualization of periodic BSR signaling messages on 2 arrived packets.

(ii) the lack of flexibility in periodic BSR. To combat these challenges, we propose an adaptive ML-enhanced approach for determining BSR periodicity based on traffic intensity prediction. By setting an appropriate periodic timer, the network can find a balance between obtaining updated buffer status information and minimizing unnecessary signaling overhead. This allows for efficient resource utilization and adaptive scheduling to address varying traffic patterns and network dynamics in 5G New Radio (NR) networks.

III. ADAPTIVE BSR FRAMEWORK

In this section, we describe our proposed framework, its implementation, and three ways how to handle UE side data for BSR periodicity prediction.

A. Framework Description

To solve the problem described in Section II, we propose an ML-based prediction method to dynamically adjust the BSR periodicity based on the historical interarrival times of the previous packets. The solution proposed in this work predicts the time between two consecutive packet arrivals at UE to adjust the BSR periodicity accordingly. The goal is to optimize the BSR periodicity and minimize overhead in terms of the number of BSR reports while ensuring timely and accurate BSR updates, as well as close to optimal end-to-end latency.

Since historical interarrival times of the previous packets are initially available at the UE side only and BSR periodicity is configured by the gNB, we provide three alternatives

on how either UE or gNB can manage this information for adaptive BSR periodicity selection.

1) Option 1. gNB infers training data

To obtain packet interarrival time data, the network (gNB) constantly monitors packet delays (which includes queuing at UE, scheduling, and transmission times). Monitoring of UL packet delays is supported at the gNB as per 3GPP specifications (TS 38.314, Sections 4.2.1.2 and 4.2.1.9 [15]). Since gNB is aware of both the time of packet reception at the gNB side and packet delay, gNB may calculate previous packet arrival time (at UE), which is usually not available at the gNB side, as follows:

$$t_{\text{arrival,UE}} = t_{\text{arrival,gNB}} - t_{\text{latency}}, \quad (1)$$

where $t_{\text{arrival,gNB}}$ is the time of the packet arrival at gNB, t_{latency} is the packet delay, and $t_{\text{arrival,UE}}$ is the packet arrival time at the UE calculated by the gNB. We note that (1) is used by the gNB to infer training data.

Hence, now the gNB can create a dataset used for training the model that consists of interarrival times of incoming data packets at UE:

$$x_i = t_{\text{arrival,UE}}(i) - t_{\text{arrival,UE}}(i-1), \quad (2)$$

where x_i is the sample in the dataset representing the number of slots between the previous $t_{\text{arrival,UE}}(i-1)$ and current $t_{\text{arrival,UE}}(i)$ packet arrivals at UE.

The gNB then uses the created dataset to train and/or retrain the prediction model. Using the trained model, gNB predicts the future traffic interarrival times translated into appropriate BSR periodicity for each UE. For this, we use the following mapping function:

$$\text{Next BSR} = \arg \min_{\eta \in \text{collection}} (|\eta - \hat{y}_i|), \quad (3)$$

which finds the closest value η to a given number $\hat{y}_i$ (predicted BSR periodicity) within a collection that represents the valid BSR periodicities.

Furthermore, the gNB periodically performs Radio Resource Control (RRC) reconfiguration to adjust the BSR periodicity of the UE based on BSR periodicity predictions. The pseudo-code for this method is provided in Algorithm 1.

2) Option 2. UE shares training data with gNB

Since the historical interarrival times of the previous packets at UE (usually available on the UE side) are used as the training dataset for the proposed method, UE needs to send to gNB the information about packet arrival periodicity. For this purpose, UE collects packet arrivals, corresponding frames, and slots, subsequently turning them into interarrival times of the incoming data packets (in slots or in milliseconds) by using (2). For the sake of convenience, we work with slots.

Before sharing that training dataset with the gNB, the dataset size should be reduced. For this purpose, the collected packet interarrival times need to be first mapped into valid

Algorithm 1: BSR periodicity selection, Option 1

```

1 gNB monitors end-to-end latency
2 gNB infers packet arrival time at UE as per (1)
3 gNB calculates packets interarrival time at UE as
  per (2)
4 if current time  $\geq$  next BSR time then
5   if current time  $\geq$  training slots and model does
     not exist then
6     gNB processes and prepares data
7     gNB trains and saves model
8   else
9     if model exists then
10      gNB processes and prepares data
11      gNB loads saved model
12      gNB does prediction of packet interarrival
        time
13      gNB converts packet interarrival time
        from slots to ms
14      gNB does mapping of packet interarrival
        time into valid BSR periodicity as
        per (3)
15      gNB performs RRC reconfiguration
16 return next BSR periodicity
17 end

```

BSR periodicity as per (3). Then, the BSR periodicity value is transformed into a 4-bit binary representation that denotes its index among the 16 indices of the values defined by 3GPP (e.g., index 0 corresponds to the first allowed BSR periodicity value of 1 ms). For that, each 2 indices can use a single byte (8 bits). Such dataset is transmitted to gNB by UE using 3GPP collaboration level y [16]. Then, the gNB can decode the 4 bit BSR indices into BSR periodicities used as input for training the ML model. Using the trained model, gNB predicts the appropriate BSR periodicity for each UE. An illustrative representation of the proposed method wherein the UE shares the dataset with the gNB is shown in Algorithm 2.

3) Option 3. UE predicts suitable BSR periodicity

Alternatively, UE might be responsible for the prediction of the next BSR periodicity. In this case, UE prepares the dataset (packet interarrival times) as indicated in Algorithm 3. Then, training and prediction of BSR periodicity are done at the UE side. Therefore, UE needs to pass predicted BSR periodicity to gNB. To this end, we propose a modified BSR structure since UE should pass the next 2 BSR periodicity indices (1 byte) to gNB within the BSR message.

As illustrated in Fig. 3, short BSR and short truncated BSR consist of 8 bits, where 3 bits are used for Logical

Algorithm 2: BSR periodicity selection, option 2

```

1 UE calculates packets interarrival time at UE as
  per (2)
2 UE converts packet interarrival time from slots to ms
3 UE does mapping of interarrival time into valid BSR
  periodicity as per (3)
4 UE transforms BSR periodicities into 16 indices of
  BSR periodicities
5 UE shares data with gNB using 3GPP collaboration
  level y
6 if  $current\ time \geq next\ BSR\ time$  then
7   if  $current\ time \geq training\ slots\ and\ model\ does$ 
      $not\ exist$  then
8     gNB processes and prepares data
9     gNB trains and saves model
10  else
11    if  $model\ exists$  then
12      gNB processes and prepares data
13      gNB loads saved model
14      gNB does the prediction of next BSR
        periodicity
15      gNB performs RRC reconfiguration
16 return next BSR periodicity
17 end

```

Algorithm 3: BSR periodicity selection, option 3

```

1 UE calculates packets interarrival time at UE as
  per (2)
2 if  $current\ time \geq next\ BSR\ time$  then
3   if  $current\ time \geq training\ slots\ and\ model\ does$ 
      $not\ exist$  then
4     UE processes and prepares data
5     UE trains and saves model
6   else
7     if  $model\ exists$  then
8       UE processes and prepares data
9       UE loads saved model
10      UE does prediction of packet interarrival
        time
11      UE converts packet interarrival time from
        slots to ms
12      UE does mapping of packet interarrival
        time into valid BSR periodicity as
        per (3)
13      UE passes the next 2 BSR indices within
        BSR message
14      gNB performs RRC reconfiguration
15 return next BSR periodicity
16 end

```

Channel Group (LCG) ID and 5 bits for buffer status report. We propose to add 8 more bits (the smallest value that can be stored in memory) so that the next two BSR periodicity indices can be transmitted to gNB. The structure of a long BSR and a long truncated BSR is as follows. The first Byte represents the LCG ID, and the other part represents the buffer status reports. The long BSR and long truncated BSR use 8 bits to allow $2^8 = 256$ BSR indices, which can accommodate a larger buffer status of 81338368 Bytes. Similarly, in this case, 8 bits must be added for the next two BSR periodicity indices.

We highlight that this option is beyond current 3GPP specifications. The rationale for altering the BSR structure in Option 3 stems from the availability of packet arrival information on the UE side, eliminating the need for UE to share data for predicting the next BSR periodicity with gNB (Option 2) or infer it (Option 1). In this case, predictions are done at the UE side and passed to the gNB during the BSR message. Hence, Option 3 represents the first effort to modify the existing protocol to enhance latency. However, protocol modifications may entail practical considerations. For example, new modifications must align with current 3GPP standards to ensure interoperability with legacy systems, which might be challenging. Moreover, any protocol modification requires approval via the 3GPP standardization process, which can be time-consuming and involve gaining consensus among various stakeholders.

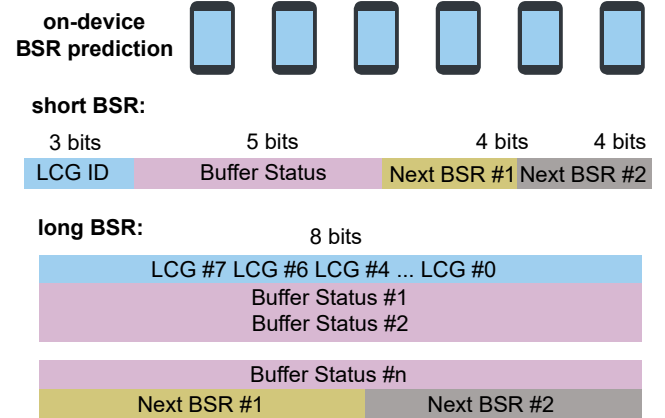


FIGURE 3. UE-side prediction: BSR structure change suggestion.

Note that the three alternatives constitute the same framework with the difference in how to handle UE side data (packet arrival times) for adjusting BSR periodicity to the UE's traffic pattern.

B. Metrics of Interest

To evaluate the performance of ML-aided adaptive BSR solution, we use such metrics as control resource usage and end-to-end latency calculated from UE MAC to gNB RLC. In this way, we are able to consider the queuing time as well as the transmission time of each packet. The number of BSRs

sent by the UE is used as the metric to evaluate the control resource usage. The preferred solution is expected to reduce the latency without incurring network resource wastage.

IV. ML FOR PACKET ARRIVAL PREDICTION

Packet arrival prediction, and traffic prediction in general, is a crucial application of ML in telecommunications. Cellular traffic prediction involves forecasting mobile network usage, understanding data traffic patterns, and optimizing network resources. In this section, we discuss the most common ML models used for traffic prediction [8], [17] and provide insights on their performance for time series forecasting of packet arrival times.

A. XGBoost

The Extreme Gradient Boosting (XGBoost) model is a powerful ML algorithm that belongs to the class of ensemble learning methods. Ensemble learning involves training and combining individual models (known as base learners) to provide a single prediction [25]. XGBoost anticipates that the base learners will be uniformly suboptimal at predicting the remainder, such that when all predictions are aggregated, inaccurate predictions counterbalance each other, while more accurate predictions accumulate to form the final, improved predictions with high accuracy.

The basic equation for the prediction made by XGBoost for a regression task is:

$$\hat{y}_i = \phi(x_i) = \sum_{k=1}^K f_k(x_i), \quad (4)$$

where $\hat{y}_i$ is the predicted output for the i -th instance, $\phi(x_i)$ represents the prediction function and x_i is the input data, K is the number of trees in the ensemble, $f_k(x_i)$ is the output of the k -th tree for the i -th instance. Note that XGBoost works by training a number of decision trees, K , and each tree k is trained on a subset of the data. The predictions from each tree are combined to form the final prediction.

The training objective can be represented as

$$\sum_{i=1}^n L(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k), \quad (5)$$

where $L(y_i, \hat{y}_i)$ represents the loss function (Mean Squared Error (MSE) in the case of regression) that calculates the difference between the predicted value $\hat{y}_i$ and the actual target value y_i for each sample, $\Omega(f_k)$ represents the regularization term that penalizes complex models to prevent overfitting.

XGBoost supports both L1 and L2 regularization:

$$\Omega(f_k) = \beta T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2, \quad (6)$$

where T is the number of leaves in the tree, w_j are the leaf weights (predicted values), β is the complexity cost per leaf, and λ is the L2 regularization coefficient. The L1 term βT encourages simpler trees with fewer leaves, while the L2 term $\frac{1}{2} \lambda \sum_{j=1}^T w_j^2$ penalizes extreme leaf weights.

Generally, the advantages of this method include the ability to detect non-linear data patterns, flexibility, as well as parallelization of tasks, which reduces training time. Among the disadvantages, one can name overfitting and sensitivity to outliers.

B. Random Forest

Random Forest is a bagging technique that contains a number of independent decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset. Instead of relying on one decision tree, the Random Forest takes the prediction from each tree and predicts the final output based on the majority votes of predictions [26].

The prediction made by a Random Forest for a regression task is represented as an aggregation/averaging of predictions from individual trees:

$$\hat{y}_i = \frac{1}{K} \sum_{k=1}^K T_k(x_i), \quad (7)$$

where $\hat{y}_i$ is the predicted output for the i -th instance, K is the number of trees in the forest, $T_k(x_i)$ represents the prediction of the k -th tree for the i -th instance.

Random Forest typically exhibits lower susceptibility to overfitting than other algorithms and maintains effectiveness even when a substantial portion of data is missing. However, the model demonstrates reduced efficiency handling time series data (compared to Support Vector Regression (SVR) and Long Short-Term Memory (LSTM)), and it may change considerably by a small variation in the data.

C. LSTM

The functioning of an LSTM network involves several formulations. We will start with the LSTM memory cell, which includes forget, input, and output gates [27]:

$$f_t = \sigma(W_f \times x_t + U_f \times h_{t-1} + b_f), \quad (8)$$

$$i_t = \sigma(W_i \times x_t + U_i \times h_{t-1} + b_i), \quad (9)$$

$$o_t = \sigma(W_o \times x_t + U_o \times h_{t-1} + b_o), \quad (10)$$

where h_t is the hidden state/output at time t , x_t is the input at time t , $W_f, W_i, W_o, U_f, U_i, U_o$ are weight matrices for respective gates, b_f, b_i, b_o are bias vectors for respective gates, σ is the sigmoid activation function, $\times$ is the matrix multiplication, and $+$ is the element-wise addition.

The equations for the cell state, candidate cell state, and the final output are:

$$\tilde{C}_t = \tanh(W_c \times x_t + U_c \times h_{t-1} + b_c), \quad (11)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \quad (12)$$

$$h_t = o_t \odot \tanh(C_t), \quad (13)$$

where C_t is the cell state at time t , W_c, U_c are weight matrices for respective gates, b_c is bias vector for respective gate, $\tanh$ is the hyperbolic tangent function, and $\odot$ is element-wise multiplication. These equations govern how an

TABLE 2. Comparison of considered ML models. ✓ means that the model is used in the study, ✗ means that it was not used.

SVR	XGBoost	Random Forest	LSTM
Advantages			
• more efficient when there is a clear margin separation between classes • efficient on non-linear problems • not biased by outliers • efficient for time series	• designed to improve training times through parallelization • can detect and learn from non-linear data patterns • flexible • scalable and fast	• less prone to overfitting than other algorithms • can be used for dimensionality reduction • effective even if large portion of data is missing	• can better handle long sequence data • avoids vanishing gradient problem • efficient for time series • not biased by outliers
Disadvantages			
• not suitable for large datasets since it takes comparatively more time to train • prone to noise • underperforms if data is not well separated	• sensitive to outliers • may overfit depending on hyper-parameters • difficulty in interpretation	• may change considerably by a small variation in the data • computations may go far more complex compared to other algorithms • not that good with time series	• high computational complexity • large number of parameters • difficulty in interpretation • long training times
[18]: traffic prediction in real wireless network			
✓ much faster training, among best performers	✗	✓ worst performance	✗
[19]: predictive modeling of high-speed links			
✓ best accuracy	✗	✗	✗
[20]: traffic flow prediction in vehicular networks			
✓ comparable performance depending on dataset	✗	✗	✓ comparable performance depending on dataset
[21]: LTE traffic flow estimation			
✓ slightly lower performance	✗	✓ best performance	✓ second best performance
[5]: buffer status report prediction			
✗	✓ second best performance	✓ best performance	✓ worst performance
[7], [8], [10]: cellular traffic prediction and classification			
✗	✗	✓ comparable performance	✓ comparable performance
[22]: cellular traffic prediction			
✗	✗	✗	✓ existing LSTM model should be improved
[23]: cellular traffic prediction			
✓ third best performance	✗	✗	✓ best performance
[24]: cellular traffic prediction			
✗	✗	✗	✓ better performance against autoregressive integrated moving average (ARIMA)

LSTM cell processes input and updates its internal state through various gates to selectively remember or forget information. The gates' outputs are controlled by learned parameters (weights and biases) that are adjusted during the training to optimize the network's performance for specific tasks.

LSTM is recognized for its efficiency in processing long sequence data and performing time series forecasting while mitigating vanishing gradient problems. Its structure is more

sophisticated (a type of recurrent neural network) compared to other algorithms. However, LSTM is characterized by high computational complexity and extended training periods, which may present challenges in scenarios requiring real-time predictions.

D. SVR

The primary objective of SVR is to maximize the number of data points within the ϵ -tube (margin). This parameter

determines the width of the tube surrounding the estimated function, also known as the hyperplane. Points falling within this tube are considered accurate predictions and are not penalized by the algorithm. Support vectors are the points that lie outside the tube, not just those on the margin. However, to prevent overfitting, some errors in the model must be accepted. This can be achieved by incorporating slack variables. The significance of these slack variables can be controlled by adjusting the regularization parameter C . Training SVR involves solving:

$$\min \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N (\xi_i + \xi_i^*), \quad (14)$$

$$\text{s.t.} \begin{cases} y_i - \langle \mathbf{w}, \phi(x_i) \rangle - b \leq \epsilon + \xi_i^*, \\ -y_i + \langle \mathbf{w}, \phi(x_i) \rangle + b \leq \epsilon + \xi_i, \\ \xi_i, \xi_i^* \geq 0, \end{cases} \quad (15)$$

where $\mathbf{w}$ is the learned weight vector, x_i is the i -th training instance, $\phi(x_i)$ is the mapping function induced by the kernel, y_i is the training label, ξ_i is the slack variable that denotes the deviation from the point to the positive edge of the hyperplane, and ξ_i^* is the slack variable that denotes the deviation from the point to the negative edge of the hyperplane. C is a constraint that controls the penalty imposed on observations outside the bounds to prevent overfitting [28].

To enable the model to make nonlinear predictions, each dot product $\langle \mathbf{w}, \phi(x_i) \rangle$ is replaced with a kernel function, effectively mapping the data into a higher-dimensional feature space. We utilize the Squared Exponential Kernel, also known as the Radial Basis Function (RBF) kernel, which is universal:

$$k_{SE}(x, x') = \exp(-\gamma \|x - x'\|^2), \quad (16)$$

where γ is the kernel coefficient, $\gamma > 0$, and x and x' are two inputs.

SVR can deal with the nonlinearities of cellular traffic, is suitable for time series predictions, and is not biased by outliers (compared to XGBoost). Regarding disadvantages, it might underperform if data is not clearly separated and might not be suitable for large datasets since it takes comparatively more time to train.

In Table 2, we provide the summary of the discussed predictors, along with examples of their performance from the literature. In brief, each estimator has its advantages and drawbacks depending on the data we feed into it. In Section V, we provide the performance analysis of described models for packet arrival time prediction on two datasets.

E. SVR with Random Fourier Features (SVR-RFF)

SVR performance can be further improved by Random Fourier Features (RFF). RFF utilize a randomized lower-dimensional feature mapping, enabling better scalability of data [29]. These constructions involve approximating the

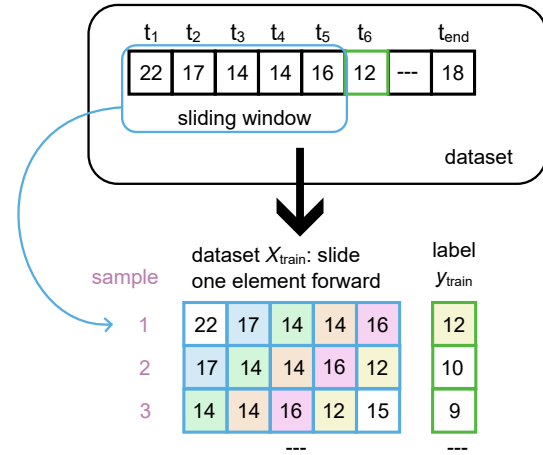


FIGURE 4. Sliding window concept illustration.

feature map denoted as:

$$\hat{\Phi} : \chi \rightarrow \mathbb{C}^R, \quad (17)$$

where R represents the space of R -dimensional complex numbers. Consequently, this leads to the definition of the approximate feature map:

$$\kappa(x, x') \approx \langle \hat{\Phi}(x), \Phi(x') \rangle_{\mathbb{C}^R}. \quad (18)$$

For the implementation, we use RBFSampler – a feature extractor employing an RBF kernel. It reshapes the input data into a new feature space with a specified dimension ($n_{\text{components}}$), achieved by either injecting random values into those features or through Monte Carlo approximation. Subsequently, these new feature vectors are utilized to train ML algorithms. The parameter γ of RBFSampler governs the width of the RBF kernel. A higher γ value produces a kernel that is more sensitive to small variations among points, whereas a lower γ value yields a kernel that is more sensitive to larger differences between points.

F. Data Augmentation

Data augmentation (originally proposed for Convolutional Neural Networks (CNNs) [30]) means increasing the variable space in such a way that a model has a higher chance of making accurate predictions/classifications. We employ the concept of the Sliding Window, a robust method used to process sequential data involving the systematic analysis of fixed-sized subsequences one at a time [31], [32]. Thus, in the preprocessed dataset X_{train} with one feature, we have window size columns, where each row is shifted by one value, and y_{train} contains one column with $(\text{window size}+1)$ -th value to be predicted (see Fig. 4 for visualization). By carefully selecting an appropriate window size, ML engineers aim to balance between granularity and generalization, allowing the model to benefit from a diverse yet focused representation of the data. This facilitates robust learning, enabling the model to generalize to unseen data

while effectively capturing the inherent complexity of the dataset.

V. SIMULATION RESULTS

In this section, we provide numerical results for the evaluation of different ML models' performance based on a publicly available dataset. We then present the results of the proposed adaptive ML-based BSR scheduling (incorporated into 5G MATLAB toolbox) versus traditional periodic BSR with fixed periodicities in terms of latency and resource usage.

A. Packet Arrival Time Prediction Using Real Traffic

In contrast to the prevailing trend in literature, which primarily focuses on predicting the size of incoming packets (traffic load), as demonstrated in [8], [10], [33], [34] and in Section I, our approach is centered on forecasting the arrival time of these packets. More specifically, our focus lies in predicting the time, measured in milliseconds/slots, when the next packet will arrive. Hence, to validate the feasibility of these predictions, we utilize a **realistic dataset** [9] published alongside [8], where surfing, video calling, voice calling, and video streaming applications are considered. This dataset captures traffic from multiple Android devices employing Virtual Private Network (VPN) tunneling. It encompasses the following features: (1) timestamps for packet arrival and departure, (2,3) source and destination IP addresses, (4) communication protocols like UDP, TCP, SSL, and (5) encrypted payload for transmissions in UL and downlink (DL) directions.

As part of the data preprocessing stage, we create a crucial feature central to our predictions: the *time difference between packet arrivals* measured in milliseconds (since the dataset initially contains milliseconds) and use only UL traffic. Once this feature is added, we divide our dataset into training and testing subsets, allocating 80% for training and 20% for testing. We utilize the following ML models: SVR, SVR-RFF, XGBoost, Random Forest, and LSTM with the following parameters. SVR is configured with squared exponential kernel, regularization parameter $C = 100$, $\gamma = 0.0001$, and $\epsilon = 0.0001$. RFF part is configured with $\gamma = 1$ and $n_{\text{components}} = 100$. XGBoost and Random Forest models use $K = 50$ estimators and a maximum depth of 5 of each estimator. LSTM structure consists of an initial normalization layer, followed by an LSTM layer with 64 units, a ReLU activation function, and dropout rates of 0.2 for both dropout and recurrent dropout. After that, there are two dense layers with 128 and 64 units, respectively, both using ReLU activation functions and a final dense layer with 1 unit. Batch size of 1 and 15 epochs are used for training. The hyperparameters of all models were tuned by selecting them empirically.

In Fig. 5, we present the accuracy of selected ML models in terms of Mean Absolute Error (MAE), 75th and 95th percentiles. Cumulative Distribution Function (CDF) of errors

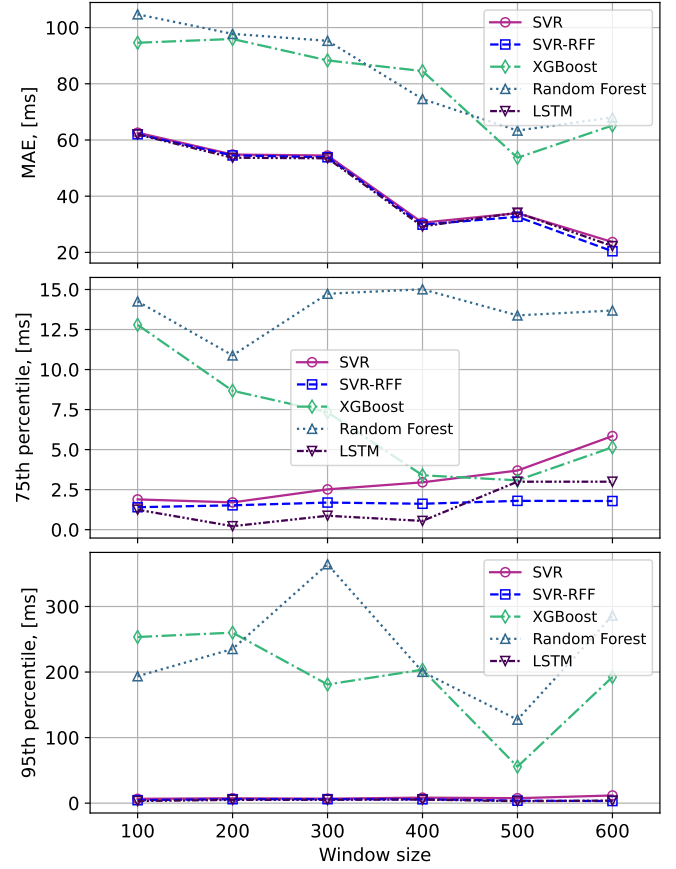


FIGURE 5. Real traffic dataset: ML models' performance vs. window size.

is depicted in Fig. 6, where the median and 95th percentile are indicated. These four metrics are calculated as:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|, \quad (19)$$

$$75 \text{ percentile} = \text{value of } \left(\frac{3(N+1)}{4} \right) \text{th ordered sample}, \quad (20)$$

$$95 \text{ percentile} = \text{value of } \left(\frac{19(N+1)}{20} \right) \text{th ordered sample}, \quad (21)$$

$$\text{Median} = \text{value of } \left(\frac{N+1}{2} \right) \text{th ordered sample}, \quad (22)$$

where N represents the total number of observations, y_i denotes the actual value of the i -th observation, and $\hat{y}_i$ is the predicted value of the i -th observation.

As shown in Fig. 5, LSTM model provides the lowest error, which means that a more complex deep learning model is suitable for this type of data corresponding to realistic traffic. SVR regressor also demonstrates comparable performance to LSTM, and SVR-RFF enhances SVR performance even more. One may notice that the variation in window size leads to a significant change in the performance of the considered ML models. This can be explained by the fact that the impact of changing the window size value depends on the specific

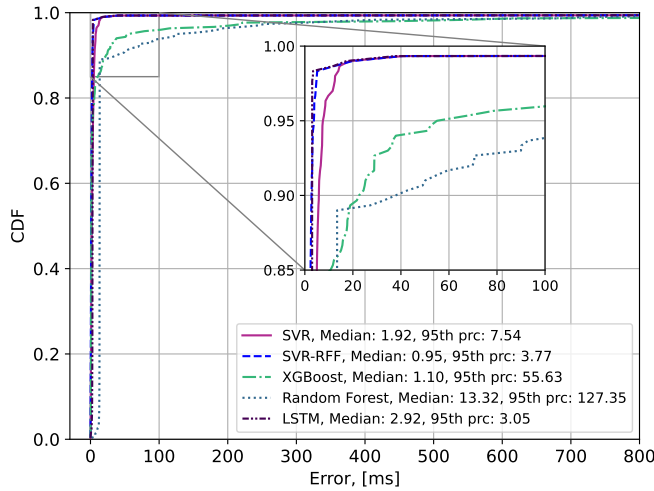


FIGURE 6. Real traffic dataset: cumulative distribution function of errors, window size = 500 samples.

algorithm (some algorithms are more sensitive to window size) and the data itself. Regarding the optimal window size value, as a general trend, MAE decreases as the window size increases (for Random Forest and XGBoost, the optimal window size is 500, while for the rest of the models, MAE keeps decreasing with an increase in window size). However, for the 95th percentile, most of the models achieve optimal results with a window size of 500. Therefore, the subsequent analysis is focused on the results for a window size of 500.

The MAE of LSTM and SVR-RFF, in this case, is 34.12 ms and 32.67 ms, respectively, which can be explained by the presence of extremely high values of the time difference between packet arrivals in the dataset. We use time difference in milliseconds, and, in some cases, the difference between packet arrivals can be a couple of seconds (which are clear outliers). When the model does not provide the exact prediction, this high value deteriorates MAE. For this reason, we present the 75th and 95th percentiles, which are 2.99 ms and 3.05 ms, respectively, for the LSTM model. It means that 95% of errors are lower than 3.05 ms, providing high accuracy. In the case of SVR-RFF, the 75th and 95th percentiles are 1.80 ms and 3.77 ms. These results demonstrate that LSTM and SVR-RFF models accurately predict in the presence of outliers compared to Random Forest and XGBoost.

To delve deeper into our findings, we introduce Fig. 6, showcasing the CDF of errors. The legend highlights the median error, a particularly robust metric, especially in scenarios influenced by outliers, and 95th percentile. One may observe that LSTM provides an accurate prediction of the time when the next packet will arrive with a median error of 2.92 ms. Moreover, LSTM shows that for 98% of predictions, the error is less than 3.36 ms. Regarding SVR-RFF, the median error is even smaller (1.92 ms), while the 98th percentile is slightly larger (5.04 ms), which can be seen from the zoomed part of the plot. Nevertheless, the results

TABLE 3. Simulation parameters

Parameter	Value
System parameters	
Number of UEs	4
Simulation time	10 s
UE position	(100 0 1.5); (250 0 1.5); (700 0 1.5); (750 0 1.5)
UL bandwidth	100 MHz
Number of RBs	237
SCS	30 kHz
UL carrier frequency	2515 MHz
Channel model	CDL-C
UE/gNB TX power	23 dBm / 29 dBm
gNB RX gain	10 dBi
Scheduler strategy	proportional fair
PUSCH preparation time	200 microseconds
HARQ processes	16
Duplex mode	FDD
Size of sub-band	16
gNB position	(0 0 10)
Traffic	FTP: file size=0.5 MBytes, exponentially distributed reading time with mean of 5ms/20ms, Poisson distribution of inter-arrival time with mean of 5ms/20ms
ML parameters	
Training dataset split	≈ 70% (670 frames)
Test dataset split	≈ 30% (300 frames)
Window size	30
SVR	kernel = squared exponential, Regularization parameter $C = 100$, kernel coefficient $\gamma = 0.1$, epsilon in epsilon-SVR model $\epsilon = 0.1$
SVR-RFF	SVR with approximation of RBF kernel feature map using RFF, $\gamma = 1$, $n_{\text{components}} = 100$
LSTM	LSTM(64, activation='relu', dropout=0.2, recurrent dropout=0.2); Dense(128, activation='relu'); Dense(64, activation='relu'); Dense(1); loss='MAE', optimizer='adam', epochs=15, batch size=1
Random Forest	$K = 50$ estimators, max depth = 5
XGBoost	$K = 50$ estimators, max depth = 5
Dataset feature	slot difference between 2 arrivals

confirm the feasibility of predicting the time differences between consecutive packet arrivals in cellular networks.

TABLE 4. FTP traffic prediction accuracy

Algorithm	MAE	Median
Accuracy in number of slots (2 slots=1 ms)		
LSTM	8.51	6.15
XGBoost	8.24	6.16
Random Forest	7.58	5.99
SVR	7.38	5.21
SVR-RFF	7.53	5.11

B. FTP Traffic with Exponential Reading Time and Poisson Inter-Arrival Time Mean of 5 ms

In this and the upcoming subsections, we evaluate our proposed adaptive ML-based BSR scheduling in terms of latency and resource usage. This evaluation is performed using MATLAB 5G Toolbox, where we generate **synthetic datasets** for the considered ML model. We assume an FTP model with a reading time that follows an exponential distribution with a mean of 5ms/20ms, and an inter-arrival time that follows a Poisson distribution with the same mean of 5ms/20ms. FTP model file size is 0.5Mbytes [35]. We implement online ML, where we predict the time difference between packet arrivals in terms of the number of slots and map it into the next BSR periodicity in milliseconds. We list the main simulation parameters in Table 3.

As a first step, we identify the ML model, among the ones discussed in Section IV, that works better for the FTP traffic. The comparison results are presented in Table 4. It is important to note that we tuned SVR $\gamma = 0.1$ and $\epsilon = 0.1$ parameters compared to Section V-A. The reason is that strict γ and ϵ in the case of FTP traffic lead to the model learning the data rather than learning its pattern. In general, predicting FTP traffic poses a greater challenge due to its inherent randomness. Therefore, it is more difficult for all considered ML models to identify the data pattern. However, SVR-RFF model achieves better performance for our FTP traffic synthetic dataset in terms of median error. We select the median as a metric that defines the optimal ML model since the distribution of errors is not symmetrical. Compared to SVR, SVR-RFF notably improves the accuracy and, hence, better suits our task.

In addition to improved accuracy, which demonstrates the model's efficiency for time-series forecasting, SVR-RFF is not influenced by outliers and exhibits lower computational complexity, for example, when compared to LSTM. Consequently, we focus our subsequent analysis on SVR-RFF for BSR periodicity prediction. Note that, in practice, the model can be pre-trained on a high number of samples and then used for predictions, or transfer learning techniques can be leveraged as an alternative [36].

Fig. 7 reports the evaluation results of our proposed adaptive ML-based BSR solution in terms of latency and resource usage for generated FTP traffic. In detail, our goal is to visualize the relationship between two metrics: latency

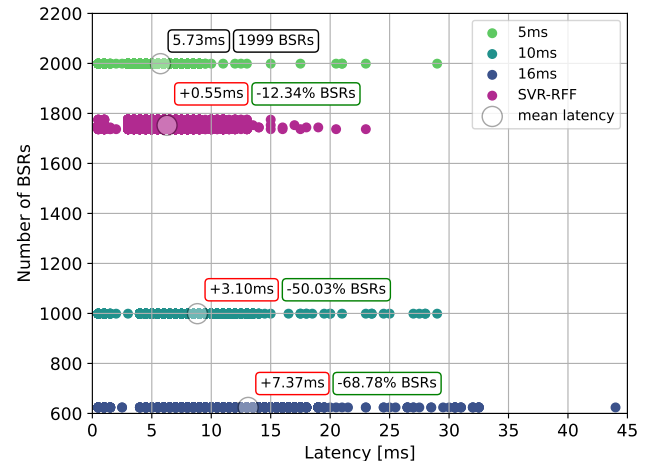


FIGURE 7. FTP traffic (exponential reading time with mean of 5 ms, and Poisson inter-arrival time with mean of 5 ms): latency vs. number of BSRs, 70%/30% train/test split.

on the x -axis and the number of sent BSRs on the y -axis. Preferably, both variables should be minimized. As can be seen, adaptive BSR periodicity by means of ML (i.e., SVR-RFF) provides flexibility in scheduling. For instance, we can see that the difference between traditional fixed BSR periodicity of 5 ms and 10 ms in terms of latency and resource performance is notable (3.10 ms and 50% of BSRs). The proposed adaptive BSR, SVR-RFF, can guarantee a latency performance close to fixed 5 ms BSR (baseline scheme for traffic arrival with mean of 5 ms) while reducing the resource utilization by 12.34%. Moreover, the scheme makes its decisions based on past traffic arrivals, which makes it adaptive to the traffic and able to save resources in case of a low traffic arrival rate. This all shows the potential of the proposed approach to be implemented within 5G feedback optimization mechanisms.

C. FTP Traffic with Exponential Reading Time and Poisson Inter-Arrival Time Mean of 20 ms

In Fig. 8, we provide FTP model traffic results using exponential reading time and Poisson inter-arrival time with a mean of 20 ms, i.e., we introduce even more randomness in the traffic arrivals. In this scenario, the fixed BSR approach with a 20 ms periodicity serves as the baseline. The results are consistent with Fig. 7 with a mean arrival rate of 5 ms. These results suggest that leveraging ML allows for adaptable resource allocation, facilitating a more suitable configuration for different interarrival times: when the mean arrival rate is 5 ms, SVR-RFF converges towards a 5 ms fixed BSR; with a mean arrival rate of 20 ms, SVR-RFF converges to a 20 ms fixed BSR periodicity. Moreover, existing 3GPP BSR implementation faces significant performance differences between 20 ms and 32 ms BSR periodicities, while the proposed method aims to dynamically adjust BSR periodicity to traffic patterns.

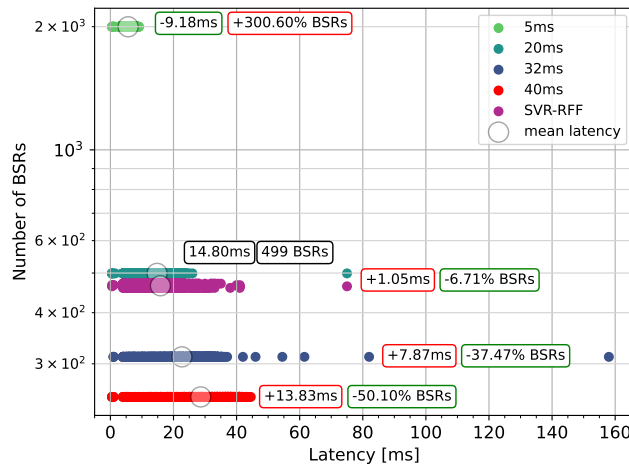


FIGURE 8. FTP traffic (exponential reading time with mean of 20 ms, and Poisson inter-arrival time with mean of 20 ms): latency vs. number of BSRs, 70%/30% train/test split.

Furthermore, in Fig. 8, one may notice the presence of outliers, for example, in the case of fixed BSR periodicity with 32 ms (see latency of around 160 ms). This can be explained by the fact that in MATLAB 5G Toolbox, we enable packet failures and in the case of retransmission, latency might increase. Recall that latency calculations encompass the time duration from the packet's arrival at RLC of the UE to its successful reception by the gNB. Consequently, standard fixed BSR periodicities of 40 ms and 32 ms notably escalate latency, while the 5 ms fixed BSR periodicity significantly increases resource utilization. Differently, the *SVR-RFF* method showcases performance closely aligned with the baseline approach, demonstrating more consistent behavior that aims to achieve a trade-off between latency and resource usage.

We also highlight that different UEs might have varying needs when it comes to sending BSRs. Some might need to send BSRs more frequently than others due to their traffic patterns. The proposed system adapts to these different requirements by using ML algorithms. The algorithm learns from the traffic patterns and behavior of each UE. This way, the system can understand which UEs require more frequent BSRs and adjust the scheduling accordingly.

D. Effect of Tunable Parameter – Train/Test Split

In this subsection, we analyze the effect of the train/test split ratio on the performance of the ML-aided BSR solutions in Fig. 9. Similarly to Fig. 7 and Fig. 8, Fig. 9 reports the latency and the number of sent BSRs. Recall that in Fig. 7 and Fig. 8, we used a 70%/30% train/test split. Here, we also add 80%/20%, 90%/10%, and 95%/5% ratios. The goal is to fine-tune this parameter and check its impact on latency and resource usage results. A larger training data size means that there are more samples for training, which leads to better accuracy. As shown in Fig. 9, a smaller test

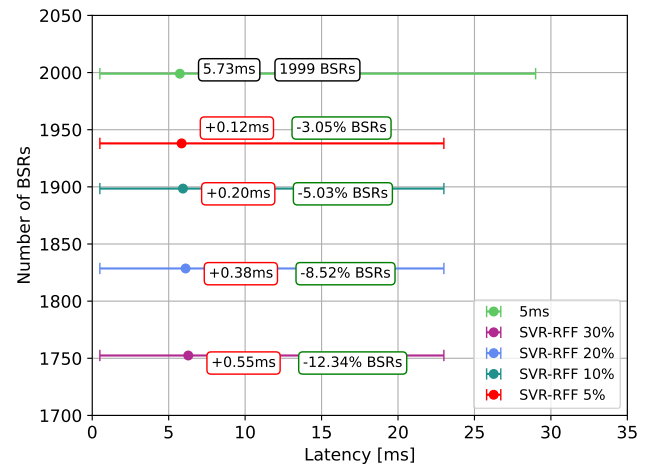


FIGURE 9. FTP traffic (exponential reading time with mean of 5 ms, and Poisson inter-arrival time with mean of 5 ms): latency vs. number of BSRs when varying train/test split ratio (test ratio is indicated in the legend).

size leads to improved latency (closer to 5 ms fixed BSR periodicity implementation) and higher usage of resources compared to the other considered test ratios.

For example, in the case of “SVR-RFF 5%”, the improvement achieved by the proposed algorithm is a reduction in resource usage by 3.05%, accompanied by a marginal increase in latency of only 2.09% (equivalent to 0.12 ms). Similarly, in the case of “SVR-RFF 30%”, performance indicates a noteworthy reduction in the number of sent BSRs by 12.34%, accompanied by a slight increase in latency of just 9.6% (equivalent to 0.55 ms). *The obtained results suggest that the train/test split parameter in our proposed solution can be adjusted according to the application requirements. In case of an application with stringent latency requirements, a smaller test size can be employed.*

VI. CONCLUSIONS AND FUTURE WORK

We proposed a framework that predicts the time between consecutive packet arrivals and adjusts the BSR periodicity accordingly. This approach adapts the BSR periodicity defined in 3GPP specifications (i.e., sf1, sf5, sf10, etc.) to the traffic arrival pattern of each user by means of ML models. Our ultimate objective is to establish a flexible scheduling framework that optimizes latency performance while conserving resources. The results demonstrate that, first, it is feasible to accurately predict the time interval between consecutive packet arrivals. Second, adaptive ML-based BSR reporting can significantly reduce the BSR reporting overhead for a small sacrifice in latency.

Furthermore, the proposed adaptive BSR scheduling framework can be implemented within 5G NR MAC layer scheduling at the gNB or the UE side. In the former case, gNB can infer the UE traffic arrival times from the end-to-end latency and reception time of the packets at gNB, or UE can share the training dataset. In the latter case, the

proposed framework can be incorporated at the UE side, which requires a slight modification of the BSR message structure.

Among the avenues for future work, the following aspects can be studied:

- Improvement of ML algorithms' robustness to avoid the need for re-training ML models each time when the traffic (i.e., data) changes.
- Determination of the optimal set of BSR periodicity values required to optimize the system performance under different conditions.
- Estimation of the computational and signaling overheads needed to incorporate ML methods in the framework.

We note that in the current work, the computational overhead associated with model training can be mitigated by deploying a pre-trained model. Regarding the signaling overhead, in the proposed Option 1, where the gNB infers UE-side data, no additional exchange of information is required. Conversely, when UE shares the training data (Option 2) or makes a prediction and then passes the next BSR indices within the modified BSR structure (Option 3), then there is a signaling overhead that is larger as compared to the traditional fixed BSR periodicity method. Therefore, it is possible to evaluate our framework's improvement in terms of both saved resources and latency vs. computational and signaling overheads.

Acknowledgments

This work was supported by the Research Council of Finland (Projects RADIANT, ECO-NEWS, and SOLID) and by Intel Corporation. The authors would like to thank Arvind Merwaday, Rath Vannithamby, and Shilpa Talwar (all with Intel Corporation, US) for their continuous support and feedback throughout this research process. Salwa Saafi was also supported by the Finnish Cultural Foundation (Grant number 00231016).

REFERENCES

- [1] F. Malandrino, C.-F. Chiasserini, and S. Kirkpatrick, "Cellular Network Traces Towards 5G: Usage, Analysis and Generation," *IEEE Transactions on Mobile Computing*, vol. 17, no. 3, pp. 529–542, 2017.
- [2] O. Chukhno, O. Galinina, S. Andreev, A. Molinaro, and A. Iera, "Interplay of User Behavior, Communication, and Computing in Immersive Reality 6G Applications," *IEEE Communications Magazine*, vol. 60, no. 12, pp. 28–34, 2022.
- [3] "Medium Access Control (MAC) Protocol Specification (Release 18)," tech. rep., 3GPP TS 38.321 V18.5.0, March 2025.
- [4] S. Kwon and N.-H. Lee, "Uplink QoS scheduling for LTE system," in *2011 IEEE 73rd Vehicular Technology Conference (VTC Spring)*, pp. 1–5, IEEE, 2011.
- [5] Q. Zhang, A. Nikou, and M. Daoutis, "Predicting Buffer Status Report (BSR) for 6G Scheduling using Machine Learning Models," in *2022 IEEE Wireless Communications and Networking Conference (WCNC)*, pp. 632–637, IEEE, 2022.
- [6] F. Ronteix-Jacquet, X. Lagrange, I. Hamchaoui, and A. Ferrieux, "Rethinking Buffer Status Estimation to Improve Radio Resource Utilization in Cellular Networks," in *2022 IEEE 95th Vehicular Technology Conference: (VTC2022-Spring)*, pp. 1–5, IEEE, 2022.
- [7] A. Azari, P. Papapetrou, S. Denic, and G. Peters, "User Traffic Prediction for Proactive Resource Management: Learning-Powered Approaches," in *2019 IEEE Global Communications Conference (GLOBECOM)*, pp. 1–6, IEEE, 2019.
- [8] A. Azari, P. Papapetrou, S. Denic, and G. Peters, "Cellular Traffic Prediction and Classification: A Comparative Evaluation of LSTM and ARIMA," in *Discovery Science: 22nd International Conference, DS 2019, Split, Croatia, October 28–30, 2019, Proceedings 22*, pp. 129–144, Springer, 2019.
- [9] A. Azari, "Cellular Traffic Analysis," <https://github.com/AminAzari/cellular-traffic-analysis>, 2019. Accessed: April 12, 2025.
- [10] A. Azari, F. Salehi, P. Papapetrou, and C. Cavdar, "Energy and Resource Efficiency by User Traffic Prediction and Classification in Cellular Networks," *IEEE Transactions on Green Communications and Networking*, vol. 6, no. 2, pp. 1082–1095, 2021.
- [11] Y. Wang and T. Nakachi, "Prediction of Network Traffic Through Light-Weight Machine Learning," *IEEE Open Journal of the Communications Society*, vol. 1, pp. 1919–1933, 2020.
- [12] C. Ren and X. Lyu, "Online-learning-based predictive optimization of uplink scheduling for industrial internet-of-things," *IEEE Open Journal of the Communications Society*, vol. 5, pp. 6817–6831, 2024.
- [13] G. Woo, C. Liu, A. Kumar, C. Xiong, S. Savarese, and D. Sahoo, "Unified Training of Universal Time Series Forecasting Transformers," 2024.
- [14] E. Dahlman, S. Parkvall, and J. Skold, *4G, LTE-Advanced Pro and the Road to 5G*. Academic Press, 2016.
- [15] "5G; NR; Layer 2 Measurements," tech. rep., 3GPP TS 38.314, V17.4.0, February 2024.
- [16] "Study on Artificial Intelligence (AI)/Machine Learning (ML) for NR air interface (Release 18)," tech. rep., 3GPP TR 38.843 V18.0.0, December 2023.
- [17] W. Jiang, "Cellular Traffic Prediction with Machine Learning: A Survey," *Expert Systems with Applications*, vol. 201, p. 117163, 2022.
- [18] D. Alekseeva, N. Stepanov, A. Veprev, A. Sharapova, E. S. Lohan, and A. Ometov, "Comparison of Machine Learning Techniques Applied to Traffic Prediction of Real Wireless Network," *IEEE Access*, vol. 9, pp. 159495–159514, 2021.
- [19] H. Ma, E.-P. Li, A. C. Cangellaris, and X. Chen, "Comparison of Machine Learning Techniques for Predictive Modeling of High-Speed Links," in *2019 IEEE 28th Conference on Electrical Performance of Electronic Packaging and Systems (EPEPS)*, pp. 1–3, IEEE, 2019.
- [20] P. Sun, N. Aljeri, and A. Boukerche, "Machine Learning-Based Models for Real-Time Traffic Flow Prediction in Vehicular Networks," *IEEE Network*, vol. 34, no. 3, pp. 178–185, 2020.
- [21] F. Yaghoubi, A. Catovic, A. Gusmao, J. Pieczkowski, and P. Boros, "Traffic Flow Estimation Using LTE Radio Frequency Counters and Machine Learning," *arXiv preprint arXiv:2101.09143*, 2021.
- [22] F. W. Alsaade and M. Hmoud Al-Adhaileh, "Cellular Traffic Prediction Based on an Intelligent Model," *Mobile information systems*, vol. 2021, pp. 1–15, 2021.
- [23] J. Feng, X. Chen, R. Gao, M. Zeng, and Y. Li, "DeepTP: An End-to-End Neural Network for Mobile Cellular Traffic Prediction," *IEEE Network*, vol. 32, no. 6, pp. 108–115, 2018.
- [24] T. Kuber, I. Seskar, and N. Mandayam, "Traffic Prediction by Augmenting Cellular Data with Non-Cellular Attributes," in *2021 IEEE Wireless Communications and Networking Conference (WCNC)*, pp. 1–6, IEEE, 2021.
- [25] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.
- [26] C. Yue, R. Jin, K. Suh, Y. Qin, B. Wang, and W. Wei, "LinkForecast: Cellular Link Bandwidth Prediction in LTE Networks," *IEEE Transactions on Mobile Computing*, vol. 17, no. 7, pp. 1582–1594, 2017.
- [27] Y. Yu, X. Si, C. Hu, and J. Zhang, "A Review of Recurrent Neural Networks: LSTM Cells and Network Architectures," *Neural Computation*, vol. 31, no. 7, pp. 1235–1270, 2019.
- [28] H. Zarini, A. Khalili, H. Tabassum, M. Rasti, and W. Saad, "AlexNet Classifier and Support Vector Regressor for Scheduling and Power Control in Multimedia Heterogeneous Networks," *IEEE Transactions on Mobile Computing*, vol. 22, no. 5, pp. 2520–2536, 2021.
- [29] A. Tompkins and F. Ramos, "Fourier Feature Approximations for Periodic Kernels in Time-Series Modelling," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, 2018.

- [30] A. Le Guennec, S. Malinowski, and R. Tavenard, "Data Augmentation for Time Series Classification Using Convolutional Neural Networks," in *ECML/PKDD Workshop on Advanced Analytics and Learning on Temporal Data*, 2016.
- [31] T. T. Um, F. M. Pfister, D. Pichler, S. Endo, M. Lang, S. Hirche, U. Fietzek, and D. Kulić, "Data Augmentation of Wearable Sensor Data for Parkinson's Disease Monitoring Using Convolutional Neural Networks," in *Proceedings of the 19th ACM international conference on multimodal interaction*, pp. 216–220, 2017.
- [32] M. M. Krell, A. Seeland, and S. K. Kim, "Data Augmentation for Brain-Computer Interfaces: Analysis on Event-Related Potentials Data," *arXiv preprint arXiv:1801.02730*, 2018.
- [33] S. Zhang, S. Zhao, M. Yuan, J. Zeng, J. Yao, M. R. Lyu, and I. King, "Traffic Prediction Based Power Saving in Cellular Networks: A Machine Learning Method," in *Proceedings of the 25th ACM SIGSPATIAL international conference on advances in geographic information systems*, pp. 1–10, 2017.
- [34] L. Yu, M. Li, W. Jin, Y. Guo, Q. Wang, F. Yan, and P. Li, "STEP: A Spatio-Temporal Fine-Granular User Traffic Prediction System for Cellular Networks," *IEEE Transactions on Mobile Computing*, vol. 20, no. 12, pp. 3453–3466, 2020.
- [35] "Study on Licensed-Assisted Access to Unlicensed Spectrum (Release 13)," tech. rep., 3GPP TR 36.889 V13.0.0, June 2015.
- [36] Y. Zhang, A. Y. Ding, J. Ott, M. Yuan, J. Zeng, K. Zhang, and W. Rao, "Transfer Learning-Based Outdoor Position Recovery with Cellular Data," *IEEE Transactions on Mobile Computing*, vol. 20, no. 5, pp. 2094–2110, 2020.



Nadezhda Chukhno is a postdoctoral researcher at Tampere University, Finland. She graduated from RUDN University, Russia, and received her B.Sc. in Business Informatics (2017) and M.Sc. in Fundamental Informatics and Information Technologies (2019). She received her double Ph.D. degree in Information Engineering (2023) from Mediterranean University of Reggio Calabria, Italy and Jaume I University, Spain, within H2020 MSCA ITN project. Her current research activity focuses on wireless communications and ML.



Salwa Saafi is a postdoctoral researcher at Tampere University, Finland. She received her Ph.D. degree (2023) in Communications Engineering from Brno University of Technology, Czech Republic and Tampere University, Finland, and her Engineering degree (2017) in Telecommunications from the Higher School of Communication of Tunis, Tunisia. Her research interests include cellular communication technologies and future wireless networks/architectures.



Sergey Andreev received the Ph.D. degree from Tampere University of Technology, and the Cand.Sc., and Dr.Habil. degrees from SUAI. He is a Professor of Wireless Communications and formerly an Academy Research Fellow with Tampere University, Finland. He is also a Research Specialist with Brno University of Technology, Brno, Czech Republic. He has been a Visiting Senior Research Fellow with King's College London, U.K., and a Visiting Postdoctoral Researcher with the University of California at Los Angeles, USA.

He co-authored more than 300 published research works on intelligent IoT, mobile communications, and heterogeneous networking.